



Multithreading Practice - CO3

[← Back](#)

QUESTIONS

Q1 ✓
Multiple Threads Using a
Single Class
10 marks**Q2** ✓
Multiple thread classes
20 marks**Q3** ✓
Thread Synchronization
10 marks

3/3 answered

Q1 Multiple Threads Using a Single Class

10 marks

Topic: Thread class, multiple threads, if-else

Question:

Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.

Armstrong → Check whether the number is an Armstrong number.

Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number

Armstrong Thread: 153 is an Armstrong number

Reverse Thread: Reverse of 153 = 351

Your Code

```
1 import java.util.Scanner;
2
3 public class NumberThread extends Thread {
4     int number;
5     String name;
6
7     NumberThread(int number, String name) {
8         this.number = number;
9         this.name = name;
10    }
11
12    public void run() {
13        if (name.equals("Prime")) {
14            boolean prime = true;
15
16            if (number < 2) {
17                prime = false;
18            } else {
19                for (int i = 2; i < number; i++) {
20                    if (number % i == 0) {
21                        prime = false;
22                        break;
23                    }
24                }
25            }
26
27            if (prime)
28                System.out.println("Prime Thread: " + number + " is a p
29            else
30                System.out.println("Prime Thread: " + number + " is not
31        }
32
33        else if (name.equals("Armstrong")) {
34            int temp = number;
35            int sum = 0;
36            int digits = 0;
37
38            while (temp > 0) {
39                digits++;
40                temp = temp / 10;
41            }
42
43            temp = number;
44
45            while (temp > 0) {
46                int digit = temp % 10;
47                int power = 1;
48
49                for (int i = 0; i < digits; i++) {
50                    power = power * digit;
51                }
52            }
53        }
54    }
55 }
```

```
32
33     else if (name.equals("Armstrong")) {
34         int temp = number;
35         int sum = 0;
36         int digits = 0;
37
38         while (temp > 0) {
39             digits++;
40             temp = temp / 10;
41         }
42
43         temp = number;
44
45         while (temp > 0) {
46             int digit = temp % 10;
47             int power = 1;
48
49             for (int i = 0; i < digits; i++) {
50                 power = power * digit;
51             }
52
53             sum = sum + power;
54             temp = temp / 10;
55         }
56
57         if (sum == number)
58             System.out.println("Armstrong Thread: " + number + " is
59         else
60             System.out.println("Armstrong Thread: " + number + " is
61     }
62
63     else if (name.equals("Reverse")) {
64         int temp = number;
65         int reverse = 0;
66
67         while (temp > 0) {
68             reverse = reverse * 10 + temp % 10;
69             temp = temp / 10;
70         }
71
72         System.out.println("Reverse Thread: Reverse of " + number +
73     }
74 }
75
76 public static void main(String[] args) throws Exception {
77     Scanner sc = new Scanner(System.in);
78
79     int number = sc.nextInt();
80
81     NumberThread t1 = new NumberThread(number, "Prime");
82     NumberThread t2 = new NumberThread(number, "Armstrong");
83     NumberThread t3 = new NumberThread(number, "Reverse");
84
85     t1.start();
86     t1.join();
87
88     t2.start();
89     t2.join();
90
91     t3.start();
92     t3.join();
93 }
94 }
```

Input

153

Output

```
Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 : 351
```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Submitted



Multithreading Practice - CO3

[← Back](#)

QUESTIONS

Q1
Multiple Threads Using a
Single Class
10 marks**Q2**
Multiple thread classes
20 marks**Q3**
Thread Synchronization
10 marks

3/3 answered

Q2 Multiple thread classes

20 marks

Question:

Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.

FactorialThread → Find the factorial of the given number.

ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.

Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number

Factorial: 7! = 5040

Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

Your Code

```
1 import java.util.Scanner;
2
3 public class PrimeThread extends Thread {
4     int n;
5     String result;
6
7     PrimeThread(int n) {
8         this.n = n;
9     }
10
11     public void run() {
12         boolean prime = true;
13
14         if (n <= 1) {
15             prime = false;
16         } else {
17             for (int i = 2; i <= n / 2; i++) {
18                 if (n % i == 0) {
19                     prime = false;
20                     break;
21                 }
22             }
23         }
24
25         if (prime)
26             result = "Prime: " + n + " is a prime number";
27         else
28             result = "Prime: " + n + " is not a prime number";
29     }
30
31     public static void main(String[] args) throws Exception {
32         Scanner sc = new Scanner(System.in);
33         int n = sc.nextInt();
34
35         PrimeThread t1 = new PrimeThread(n);
36         FactorialThread t2 = new FactorialThread(n);
37         ReverseThread t3 = new ReverseThread(n);
38
39         t1.start();
40         t2.start();
41         t3.start();
42
43         t1.join();
44         t2.join();
45         t3.join();
46
47         System.out.println(t1.result);
48         System.out.println(t2.result);
49         System.out.println(t3.result);
50     }
51 }
```

```
35 PrimeThread t1 = new PrimeThread(n);
36 FactorialThread t2 = new FactorialThread(n);
37 ReverseThread t3 = new ReverseThread(n);
38
39 t1.start();
40 t2.start();
41 t3.start();
42
43 t1.join();
44 t2.join();
45 t3.join();
46
47 System.out.println(t1.result);
48 System.out.println(t2.result);
49 System.out.println(t3.result);
50 }
51 }
52
53 class FactorialThread extends Thread {
54     int n;
55     String result;
56
57     FactorialThread(int n) {
58         this.n = n;
59     }
60
61     public void run() {
62         long fact = 1;
63
64         for (int i = 1; i <= n; i++) {
65             fact = fact * i;
66         }
67
68         result = "Factorial: " + n + " ! = " + fact;
69     }
70 }
71
72 class ReverseThread extends Thread {
73     int n;
74     String result;
75
76     ReverseThread(int n) {
77         this.n = n;
78     }
79
80     public void run() {
81         int temp = n;
82         int reverse = 0;
83
84         while (temp != 0) {
85             reverse = reverse * 10 + temp % 10;
86             temp = temp / 10;
87         }
88
89         result = "Reverse: Reverse of " + n + " = " + reverse;
90     }
91 }
```

Input

7

Output

```
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7
```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



20.00 / 20 marks

 Submitted



Multithreading Practice - CO3

[← Back](#)

QUESTIONS

Q1 ✓
Multiple Threads Using a
Single Class
10 marks**Q2** ✓
Multiple thread classes
20 marks**Q3** ✓
Thread Synchronization
10 marks

3/3 answered

Q3 Thread Synchronization

10 marks

opic: synchronized, shared resource, race condition

Question:

Write a Java program to demonstrate thread synchronization using a shared Counter object.

Create a class Counter containing an integer variable count, initially set to 0.

Create two threads, ThreadA and ThreadB. Both threads should increment the same count variable 1000 times.

Implement the increment() method using the synchronized keyword so that only one thread can modify the counter at a time.

After both threads complete their execution, display the final value of count.

Expected Output:

ThreadA completed

ThreadB completed

Final Count = 2000

Requirements:

Use a shared Counter object.

Both threads must access the same Counter object.

Use synchronized for the increment operation.

Use join() to ensure both threads finish before displaying the final count.

Your Code

```
1 class Counter {
2     int count = 0;
3
4     synchronized void increment() {
5         count++;
6     }
7
8     public static void main(String[] args) throws Exception {
9         Counter counter = new Counter();
10
11         ThreadA threadA = new ThreadA(counter);
12         ThreadB threadB = new ThreadB(counter);
13
14         threadA.start();
15         threadB.start();
16
17         threadA.join();
18         threadB.join();
19
20         System.out.println("ThreadA completed");
21         System.out.println("ThreadB completed");
22         System.out.println("Final Count = " + counter.count);
23     }
24 }
25
26 class ThreadA extends Thread {
27     Counter counter;
28
29     ThreadA(Counter counter) {
30         this.counter = counter;
31     }
32
33     public void run() {
34         for (int i = 0; i < 1000; i++) {
35             counter.increment();
36         }
37     }
38 }
39
40 class ThreadB extends Thread {
41     Counter counter;
42
43     ThreadB(Counter counter) {
44         this.counter = counter;
45     }
46 }
```

Use join() to ensure both threads finish before displaying the final count.

Your Code

```
1 class Counter {
2     int count = 0;
3
4     synchronized void increment() {
5         count++;
6     }
7
8     public static void main(String[] args) throws Exception {
9         Counter counter = new Counter();
10
11         ThreadA threadA = new ThreadA(counter);
12         ThreadB threadB = new ThreadB(counter);
13
14         threadA.start();
15         threadB.start();
16
17         threadA.join();
18         threadB.join();
19
20         System.out.println("ThreadA completed");
21         System.out.println("ThreadB completed");
22         System.out.println("Final Count = " + counter.count);
23     }
24 }
25
26 class ThreadA extends Thread {
27     Counter counter;
28
29     ThreadA(Counter counter) {
30         this.counter = counter;
31     }
32
33     public void run() {
34         for (int i = 0; i < 1000; i++) {
35             counter.increment();
36         }
37     }
38 }
39
40 class ThreadB extends Thread {
41     Counter counter;
42
43     ThreadB(Counter counter) {
44         this.counter = counter;
45     }
46
47     public void run() {
48         for (int i = 0; i < 1000; i++) {
49             counter.increment();
50         }
51     }
52 }
```

Input

Output

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Visible Test Cases

- | | |
|-------------|---|
| Test Case 1 | ✓ |
| Test Case 2 | ✓ |
| Test Case 3 | ✓ |

10.00 / 10 marks

Submitted